DocRenew

Functional Specification Document

Employee Document Renewal Management System

Field	Detail
Document title	Functional Specification Document — DocRenew
Version	1.0 (Draft)
Date	13 October 2026
Prepared by	Anil B, Angular Developer — Zamorins Solutions
Related document	Business Requirements Document (BRD) v1.0, 17 September 2026
Contact	contact@zamorinstech.com
Status	Draft — for technical & stakeholder review

Table of Contents

1. Introduction

2. System Architecture

3. User Roles & Permissions

4. Functional Specifications

5. Notification & Escalation Design

6. Data Model

7. API Design

8. UI/UX Specifications

9. Technical Stack

10. Security Specifications

11. Non-Functional Requirements

12. Testing Strategy

13. Deployment & Operations

14. Out of Scope (Phase 1)

15. Assumptions & Dependencies

16. Revision History

1. Introduction

1.1 Purpose

This Functional Specification Document (FSD) defines what the DocRenew system will do and how it will do it, from both a functional and technical standpoint. It is the primary reference for architects, developers, QA engineers, and stakeholders validating that the built system matches business intent. It translates the Business Requirements Document (BRD v1.0, 17 September 2026) into concrete system design, so that engineering and QA can build and verify against a single, unambiguous specification.

1.2 Background

Organizations employing staff who hold time-bound documents — passports, visas, work permits, driving licenses, and professional certifications — face legal, regulatory, and operational risk when these documents expire unnoticed. The same risk extends to time-bound organizational items tracked just as poorly today — product warranties, vehicle maintenance schedules, recurring bills, and Annual Maintenance Contracts (AMCs). Manual tracking (spreadsheets, email chains, memory) does not scale and fails silently for either category.

DocRenew replaces this with a proactive, automated, cloud-based SaaS platform that:

- Stores every employee document record — and warranty, vehicle maintenance, bill, and AMC record — in a single, secure, central repository.
- Automatically sends escalating reminders to both the employee and HR/company in parallel at defined intervals before and after expiry.
- Delivers notifications via email, SMS, and mobile/web push.
- Provides a live, real-time dashboard of document status across the entire workforce.
- Offers role-based access (Admin, HR, Manager, Employee) so each user sees the right data.
- Generates audit-ready compliance reports on demand.

1.3 Scope — In Scope (Phase 1)

- Central, secure repository for employee document records and metadata (type, number, issue date, expiry date, optional attachments).
- Extended item categories using the same repository and metadata model: product warranties, vehicle maintenance, recurring bills, Annual Maintenance Contracts (AMCs), and a configurable Other Notifications category.
- Automated, escalating notification engine covering five reminder stages: 3 months, 1 month, 5 days, on expiry, and after expiry (management escalation).
- Multi-channel notification delivery: email, SMS, mobile/web app push.
- Live dashboard showing upcoming, expiring soon, expired, and recently renewed documents, with filtering, sorting, and bulk actions.
- Role-based access control for four roles: Admin, HR, Manager, Employee.
- Configurable reminder schedules and rules per document type and per organization.
- Document status tracking and compliance reporting (audit-ready PDF/CSV reports on demand).
- Web application (primary interface) and mobile application (PWA with push notification support).
- Employee profile management (HR/Admin create/edit; employees update own profile).
- User account lifecycle: invitation → activation → active → suspension.

- Tenant-level configuration: branding, timezone, date format, reminder schedules, document types.

1.4 Scope — Out of Scope (Phase 1)

- Automated verification or renewal submission with third-party government/regulatory portals.
- Payroll, benefits, or full HRIS functionality beyond document tracking.
- Native offline mode for mobile applications (read-only cache via PWA service worker only).
- Integration with specific third-party HRIS/ERP systems (assessed for a later phase).
- AI/ML-based document expiry prediction or document image OCR/auto-extraction (deferred).

1.5 Glossary

Term	Definition
Tenant	A single organization using the DocRenew SaaS, with its own users, documents, configuration, and data isolation.
Document	A compliance record (passport, visa, license, etc.) with type, number, issue date, expiry date, optional attachment, and computed status.
Extended Item	A non-document, time-bound record tracked on the same reminder engine as a Document — warranty, vehicle maintenance, bill, or AMC — with a due date, optional attachment, and computed status.
Employee Profile	A record representing one workforce member, linked to document records and (optionally) a user account.
User Account	A sign-in identity (email + password) with a role, linked to at most one employee profile.
Reminder Stage	One slot in the notification schedule: early (90d), follow-up (30d), urgent (5d), expiry (0d), escalation (post-expiry).
Escalation	Routing an unresolved, expired document to management via notification and dashboard visibility.
Notification Channel	The delivery medium: email, SMS, or push notification.
RBAC	Role-Based Access Control — restricting system access based on the authenticated user's role.
Audit Report	A compliance document (PDF/CSV) listing document inventory, renewal status, overdue items, and/or system activity for regulatory review.
Scan Window	The time window around a reminder trigger offset within which the notification engine will fire; avoids missing triggers due to scan interval gaps.

2. System Architecture

2.1 High-Level Architecture

DocRenew is a multi-tenant, cloud-native SaaS platform using a layered architecture. Each layer has a distinct responsibility, and components within a layer are stateless to enable horizontal scaling.

Layer	Components	Responsibility
Client	Web App (Angular SPA), Mobile App (PWA + push), Admin Console	All communicate via HTTPS/WSS through the API Gateway.
API Gateway	Single entry point	TLS termination, rate limiting, JWT validation, routing, request/response logging.
Application	Auth, Document, Notification, Dashboard, Reporting, Profile, Config, Scheduling services	Eight stateless backend services, one per bounded context.
Data	PostgreSQL, S3-compatible object store, Redis	Primary DB, encrypted attachment storage, cache/queue/job locks.
External Integrations	Email provider, SMS gateway, Push bridge (FCM/APNS)	All behind provider-agnostic adapters.
Scheduling	Background scan jobs + event hooks	Default 15-min scan, plus event-driven re-evaluation on document create/update/expiry change.

2.2 Architecture Principles

- Multi-tenant isolation by design. Every query is scoped by tenantId from the authenticated session; enforced at the application layer and audited.
- Stateless application services. All state lives in the database or cache, so any service instance is interchangeable for horizontal scaling.
- Scheduled + event-driven hybrid. The notification engine runs on a configurable scan interval (default 15 min) as the primary path, with event-driven re-evaluation hooks as an optimization.
- Graceful degradation. If one notification channel fails (e.g. SMS gateway down), other channels still deliver; failures are logged and surfaced for review.
- Configuration over code. Reminder schedules, document types, notification templates, and role permissions are configurable per tenant without code changes.
- Defense in depth. Encryption at rest and in transit, RBAC at every layer, signed attachment URLs with short TTLs, and audit logging of sensitive operations.

2.3 Component Responsibilities

Component	Responsibility
Web App (Angular SPA)	Primary interface for all roles; full feature access via browser; responsive, role-adaptive layouts.
Mobile App (PWA)	On-the-go document viewing, reminder acknowledgment, push notifications; service worker caches read-only data for offline viewing.

Component	Responsibility
API Gateway	TLS termination, per-user/per-tenant rate limiting, routing, logging, JWT validation and tenant-context extraction.
Auth Service	Authentication (email/password + JWT), session management, role resolution, password reset, account invitation/activation.
Document Service	Document CRUD, expiry tracking, status computation, attachment URL signing, reminder re-evaluation trigger on change.
Notification Service	Scans for due reminders, renders templates, dispatches via email/SMS/push adapters, tracks delivery status and retries.
Dashboard Service	Aggregates document status counts/lists per role scope (tenant, team, personal); filtering, sorting, pagination, bulk actions.
Reporting Service	Generates audit-ready PDF/CSV reports on demand; queues large report jobs; stores generated reports for retrieval.
Profile Service	Employee profile CRUD, manager-relationship management, notification preferences, profile-to-account linking.
Configuration Service	Tenant settings CRUD; reminder schedule definitions (base + per-type overrides); document type and template management.
Scheduling Service	Manages background scan jobs; configurable interval; timezone-aware; job locking; failed-job retry.

2.4 Multi-Tenancy Model

DocRenew uses a single-instance, multi-tenant model with discriminator-column isolation:

- Every tenant-scoped table has a tenantId column (UUID, NOT NULL, indexed); all queries include WHERE tenantId = :currentTenantId, enforced by the ORM/query layer and verified in integration tests.
- Users belong to exactly one tenant — no cross-tenant user sharing in Phase 1.
- Configuration (reminder schedules, document types, branding, templates) is stored per tenant.
- JWTs carry tenantId and role claims; downstream services validate both on every request.
- Isolation is enforced at the application layer and audited in the query log for compliance review.

A new tenant is provisioned by: creating a tenant record and branding; creating the initial Admin account (invited via email); applying default configuration (reminder schedule, document types, templates, timezone); after which the Admin customizes settings and invites HR users and employees.

2.5 Notification Processing Model

- Scheduled scan (primary path): a background job runs every N minutes (default 15, configurable per tenant). It finds documents whose expiryDate minus the stage offset falls within the scan window of any active reminder stage, and — where no duplicate notification exists for that document+stage — creates and dispatches a notification per configured channel.
- Event-driven re-evaluation (optimization): creating, updating, or changing a document's expiryDate immediately re-evaluates its reminder schedule and enqueues any due notifications, avoiding the scan-interval lag.
- Timezone handling: reminder offsets are computed relative to the tenant's configured timezone, so a '5-day' reminder fires at the correct local time regardless of where backend hosts run.

Scan window: the time range around the exact trigger offset within which the engine will fire a reminder. With a 15-minute scan and a ±7-minute window, any document whose trigger time falls within that 14-minute span is picked up in that cycle. Default: ±7 minutes (half the scan interval), configurable per tenant.

2.6 Technology Stack Overview

Layer	Stack
Frontend	Angular 17+ (standalone components, signals), TypeScript 5.x, RxJS, Angular Material base, SCSS theming, PWA service worker for mobile.
Backend	Node.js + TypeScript, NestJS (modular services), JWT auth via passport-jwt, class-validator, Swagger/OpenAPI auto-docs, provider SDK adapters.
Database	PostgreSQL 15+ (primary), JSONB for config/templates, pgcrypto for field-level encryption, PgBouncer pooling, Redis for cache/queue/locks.
Infrastructure	Cloud-hosted (AWS/Azure/GCP), Docker containers, CI/CD via GitHub Actions, managed Postgres/Redis, S3-compatible storage, monitoring & alerting.

3. User Roles & Permissions

3.1 Role Definitions

Role	Description
Admin	Full system configuration access within the tenant: manages org settings, reminder rules, document types, users, and roles.
HR	Manages employee document records across the tenant, monitors the compliance dashboard, and follows up on renewals.
Manager	Views the compliance status of their direct reports (resolved via the employee profile's managerId) and receives escalation notifications for their team.
Employee	Views and manages their own document records and receives personal renewal reminders.

3.2 Permission Matrix

Module / Action	Admin	HR	Manager	Employee
Tenant configuration (reminder rules, doc types, branding)	Full	View	None	None
User & role management	Full	Invite/Edit	None	None
Employee profiles — all employees	Full	Full	View (own team)	None
Employee profiles — own profile	Full	Full	Full	Edit (limited fields)
Document records — all employees	Full	Full	View (own team)	None
Document records — own documents	Full	Full	Full	Full
Live dashboard	Tenant-wide	Tenant-wide	Team scope	Own scope
Compliance reports	Full	Full	Team scope	None
Notification templates & schedules	Full	View	None	None
Audit log	Full	View	None	None

3.3 Manager Scope Resolution

A Manager's visible data set is resolved at query time from employee_profiles.managerId: the dashboard, document list, and reports queries are automatically scoped to the set of employee profiles where managerId equals the authenticated Manager's own profile id, in addition to the standard tenantId scope. Multi-level hierarchies (a Manager's Manager) are out of scope for Phase 1 — Manager scope is direct reports only.

3.4 Authentication & Session Management

- Authentication: email + password against the Auth Service, returning a short-lived JWT access token plus a refresh token.

- Session: access tokens expire after 30 minutes; refresh tokens after 14 days of inactivity. Both are revocable (e.g. on password change or admin-forced logout).
- Password policy: minimum 10 characters, at least one number and one letter; passwords are hashed with bcrypt (cost factor 12).
- Account lifecycle: Invited (email sent, account inactive) → Activated (user sets password) → Active → Suspended (Admin/HR action; login blocked, data retained).
- Password reset: time-limited, single-use reset token emailed to the account's registered address.

4. Functional Specifications

4.1 Document Repository

Core fields on a document record:

Field	Type	Notes
documentType	Reference (document_types)	Passport, Visa, Work Permit, Driving License, Certification, Warranty, Vehicle Maintenance, Bill, AMC, Other Notification, or a tenant-defined type.
documentNumber	String, encrypted at rest	Identifying number; masked in list views, shown in full only on explicit reveal with an audit entry.
issueDate	Date	Optional for some document types.
expiryDate	Date, required	Drives all reminder-schedule calculations.
attachment	File reference (object store key)	Optional scanned copy; access via short-lived signed URL only.
status	Computed enum	Valid / Expiring Soon / Expiring Urgent / Expired / Renewed — derived from expiryDate and today's date in the tenant's timezone.
ownerEmployeeId	Reference (employee_profiles)	The employee the document belongs to.

Business rules:

- A document's status is recomputed on every read (not stored as a mutable field) so it is always accurate relative to the current date.
- Marking a document 'Renewed' requires entering a new expiry date (and optionally a new document number/attachment); the system records the renewal event and resets the notification schedule against the new expiry date.
- HR/Admin can bulk-import documents via CSV; each row is validated (required fields, valid date formats, known document type) before commit, with a per-row error report for failed rows.
- Document attachments are virus-scanned on upload before being accepted into the object store.

4.2 Notification Engine

See Section 5 for the full notification and escalation design. Functionally, the engine must:

- Evaluate every active document against its tenant's reminder schedule on each scan cycle.
- Avoid duplicate sends: at most one notification per document, per stage, per channel.
- Record every attempt (queued, sent, delivered, failed, bounced) with timestamps for audit and troubleshooting.
- Retry transient delivery failures with exponential backoff, up to a configurable maximum attempt count.
- Continue delivering on unaffected channels if one channel's provider is unavailable.

4.3 Dashboard & Reporting

Dashboard views, scoped per role (Section 3.2):

- Summary tiles: counts of Valid, Expiring Soon (≤90d), Expiring Urgent (≤5d), Expired, and Renewed (last 30d) documents in scope.
- Filterable/sortable document list: by employee, document type, status, expiry date range, and department (where applicable).
- Bulk actions: export selected rows, send a manual reminder, reassign owner (Admin/HR only).

Reports (generated on demand, PDF or CSV):

- Compliance snapshot — full document inventory with current status, as of the report date.
- Overdue/expired report — all documents past expiry, with days overdue and escalation status.
- Notification activity report — delivery log for a selected date range, by channel and outcome.
- Renewal history report — all renewal events in a selected period, for audit trail purposes.

Reports above a configurable row-count threshold are generated asynchronously by the Reporting Service and made available for download once complete, rather than blocking the requesting page.

4.4 Profile & User Management

Employee profile fields: name, employee ID, department, job title, manager (link to another profile), contact email/phone, and linked user account (optional — a profile can exist without system login access, e.g. for contractors tracked by HR only).

- HR/Admin create and edit all employee profiles; Employees may edit their own contact details only.
- Linking a profile to a user account sends an invitation email; the employee sets their password to activate.
- Deactivating an employee suspends their user account and retains their document history for audit purposes; it does not delete records.

4.5 Configuration Management

- Reminder schedule: tenant-wide default stages (90d / 30d / 5d / 0d / post-expiry), with optional per-document-type overrides.
- Document types: tenant can add, rename, reorder, or retire document types; retiring a type does not delete existing documents of that type.
- Notification templates: one template per stage per channel per tenant, with merge fields (employee name, document type, expiry date, days remaining); a default template set ships out of the box.
- General settings: branding (logo, name), timezone, date format, default scan interval and scan window.

5. Notification & Escalation Design

The notification engine runs a proven, escalating reminder timeline against every tracked document's expiry date:

Stage	Offset	Recipients	Default channels	Purpose
Early reminder	3 months before	Employee & HR	Email	Begin the renewal process early.
Follow-up	1 month before	Employee & HR	Email	Follow up if renewal is still pending.
Urgent	5 days before	Employee & HR	Email + SMS	Urgent alert to avoid expiration.
Expiry	On expiry date	Employee & HR	Email + SMS + Push	Document has expired — immediate action required.
Escalation	After expiry (recurring)	Management	Email + Push	Escalation for overdue, unresolved renewals.

Design rules:

- Escalation is not a one-time event: if a document remains unrenewed after the expiry stage, the escalation notification repeats on a configurable cadence (default: every 7 days) until the document is renewed or the role responsible dismisses it with a reason.
- Each notification record stores: documentId, stage, channel, recipient, status (Queued/Sent/Delivered/Failed/Bounced), provider message id, and timestamps for each status transition.
- Delivery failures retry with exponential backoff (1 min, 5 min, 30 min, 2 hr) up to 4 attempts, after which the notification is marked Failed and surfaced on an Admin-visible delivery-issues list.
- All stage offsets, recipient rules, and channels are configurable per tenant and per document type in Configuration Management (Section 4.5).

6. Data Model

6.1 Entity Relationship Overview

- tenants (1) — (many) users, employee_profiles, documents, document_types, reminder_schedules, notification_templates, audit_logs: every tenant-scoped table carries a tenantId foreign key.
- employee_profiles (1) — (many) documents: each document belongs to exactly one employee profile.
- employee_profiles (1) — (0..1) users: a profile may optionally be linked to a sign-in account.
- employee_profiles (many) — (0..1) employee_profiles via managerId: self-referencing manager relationship.
- documents (many) — (1) document_types: each document has one type, defined per tenant.
- documents (1) — (many) notifications: every dispatched or queued notification references the document and the stage that triggered it.
- reminder_schedules (1) — (many) documents, scoped by tenant and optionally by document_type: defines the offsets used for Section 5's stages.

6.2 Table Specifications

tenants

One row per customer organization.

Column	Type	Notes
id	UUID (PK)	
name	varchar	Organization name.
timezone	varchar	IANA timezone id, used for all reminder offset calculations.
settings	jsonb	Branding, date format, default scan interval/window.
createdAt	timestamp	

users

Sign-in identities.

Column	Type	Notes
id	UUID (PK)	
tenantId	UUID (FK → tenants)	Indexed; enforced on every query.
email	varchar, unique per tenant	
passwordHash	varchar	bcrypt hash.
role	enum	Admin / HR / Manager / Employee.
employeeProfileId	UUID (FK → employee_profiles, nullable)	

Column	Type	Notes
status	enum	Invited / Active / Suspended.

employee_profiles

Workforce members tracked by the system.

Column	Type	Notes
id	UUID (PK)	
tenantId	UUID (FK → tenants)	
name	varchar	
employeeCode	varchar	Org-assigned employee ID.
department	varchar	
managerId	UUID (FK → employee_profiles, nullable)	Self-referencing; drives Manager scope (Section 3.3).
contactEmail / contactPhone	varchar	
active	boolean	false on deactivation; history retained.

document_types

Tenant-defined document and extended-item categories.

Column	Type	Notes
id	UUID (PK)	
tenantId	UUID (FK → tenants)	
name	varchar	e.g. Passport, Visa, Work Permit, Warranty, Vehicle Maintenance, Bill, AMC, Other Notification.
category	enum: EMPLOYEE_DOCUMENT | EXTENDED_ITEM	Distinguishes employee-tied documents from organizational items (warranty, vehicle, bill, AMC, other).
sortOrder	integer	
retired	boolean	Hides from new-document forms without deleting history.

documents

The core compliance record — covers both employee documents and extended items (warranty, vehicle maintenance, bills, AMC, other).

Column	Type	Notes
id	UUID (PK)	

Column	Type	Notes
tenantId	UUID (FK → tenants)	
employeeProfileId	UUID (FK → employee_profiles, nullable)	Required for EMPLOYEE_DOCUMENT; optional for an EXTENDED_ITEM owned by the organization rather than a person (e.g. a company vehicle or office AMC).
documentTypeId	UUID (FK → document_types)	
category	enum: EMPLOYEE_DOCUMENT | EXTENDED_ITEM	Denormalized from document_types for fast filtering.
itemLabel	varchar, nullable	Free-text label identifying the item when there's no employee to name it by, e.g. "Company Car KA01AB1234", "Office AC — AMC".
documentNumber	varchar, encrypted	
issueDate	date, nullable	
expiryDate	date	Drives the reminder schedule; for extended items this is the renewal/due date.
attachmentKey	varchar, nullable	Object-store key; never the file itself.
lastRenewedAt	timestamp, nullable	

reminder_schedules

Per-tenant (and optionally per-document-type) stage offsets.

Column	Type	Notes
id	UUID (PK)	
tenantId	UUID (FK → tenants)	
documentTypeId	UUID (FK → document_types, nullable)	Null = tenant-wide default.
stages	jsonb	Array of {stage, offsetDays, channels, recipients}.

notification_templates

Rendered message content per stage/channel.

Column	Type	Notes
id	UUID (PK)	
tenantId	UUID (FK → tenants)	
stage	enum	Early / Follow-up / Urgent / Expiry / Escalation.
channel	enum	Email / SMS / Push.
body	text	Template with merge fields.

notifications

Delivery log — one row per attempted send.

Column	Type	Notes
id	UUID (PK)	
tenantId	UUID (FK → tenants)	
documentId	UUID (FK → documents)	
stage	enum	
channel	enum	
status	enum	Queued / Sent / Delivered / Failed / Bounced.
providerMessageId	varchar, nullable	
attemptCount	integer	
createdAt / updatedAt	timestamp	

audit_logs

Record of sensitive actions for compliance review.

Column	Type	Notes
id	UUID (PK)	
tenantId	UUID (FK → tenants)	
actingUserId	UUID (FK → users)	
action	varchar	e.g. document.viewNumber, user.suspend, config.update.
entityType / entityId	varchar / UUID	
createdAt	timestamp	

7. API Design

7.1 API Principles

- RESTful JSON API, versioned at /api/v1.
- Authentication via Bearer JWT in the Authorization header; tenantId and role are derived from the token, never trusted from the request body.
- Pagination via ?page & ?pageSize query parameters, with { data, page, pageSize, total } response envelopes for list endpoints.
- Errors follow a consistent shape: { error: { code, message, details? } }, with standard HTTP status codes (400 validation, 401 unauthenticated, 403 forbidden, 404 not found, 409 conflict).
- All endpoints are documented via Swagger/OpenAPI, generated from source annotations.

7.2 API Endpoints

Module	Endpoint	Description
Auth	POST /auth/login	Authenticate and receive access + refresh tokens.
Auth	POST /auth/refresh	Exchange a refresh token for a new access token.
Auth	POST /auth/password-reset	Request / complete a password reset.
Documents	GET /documents	List documents, filterable by status, type, employee, date range; scoped by role.
Documents	POST /documents	Create a document record.
Documents	PATCH /documents/:id	Update a document, including renewal (new expiry date).
Documents	GET /documents/:id/attachment-url	Issue a short-lived signed URL for the attachment.
Notifications	GET /notifications	Query the delivery log by document, stage, status, date range.
Notifications	POST /notifications/:id/retry	Manually retry a failed notification.
Dashboard	GET /dashboard/summary	Role-scoped status tile counts.
Reports	POST /reports	Request a report generation job (compliance, overdue, activity, renewal-history).
Reports	GET /reports/:id	Poll status / retrieve the generated report file.
Profiles	GET/POST/PATCH /employee-profiles	Employee profile CRUD.
Config	GET/PATCH /config/reminder-schedules	Manage reminder stage offsets and overrides.
Config	GET/POST/PATCH /config/document-types	Manage tenant document types.
Config	GET/PATCH /config/notification-templates	Manage per-stage, per-channel message templates.

8. UI/UX Specifications

8.1 Design System & Theme

Visual identity carried over from the DocRenew pitch materials: navy (#0B2545) and blue (#1D6FB8) as primary colors, teal (#12A4A4) as the accent, with semantic status colors for green (valid/renewed), amber (expiring soon/urgent), and red (expired/overdue). Component library: Angular Material as the base, themed to this palette, with a custom component layer for status pills, timeline steppers, and dashboard tiles.

8.2 Page Layouts & Components

Role	Key screens
Admin	Tenant settings, document types, reminder schedules, notification templates, user & role management, audit log.
HR	Live dashboard (tenant-wide), document list & detail, employee profile management, bulk import, reports.
Manager	Team dashboard (direct reports only), team document list, escalation alerts for their team.
Employee	My documents, my profile, notification preferences, upload/renew flow for own documents.

8.3 User Flows

Document renewal reminder flow

Document approaches expiry → scheduled scan matches a reminder stage → notification(s) dispatched to employee and HR → employee/HR updates the document with a new expiry date (and optionally a new attachment) → status recalculates to Valid → reminder schedule resets against the new expiry date.

New employee onboarding flow

HR creates an employee profile → HR adds the employee's tracked documents with expiry dates → (optional) HR links the profile to a user account, triggering an invitation email → employee activates their account and can view/manage their own documents.

Escalation flow

Document passes its expiry date without renewal → on-expiry notification fires to employee and HR → if still unrenewed, the escalation stage begins, notifying the employee's Manager on a recurring cadence → Manager/HR resolves by renewing the document or dismissing the escalation with a recorded reason, which is retained in the audit log.

9. Technical Stack

The full stack is detailed in Section 2.6. In addition to the runtime stack, the following supporting tools apply across the codebase:

- Testing: Jest (unit, backend and frontend), Playwright (end-to-end), Supertest (API integration tests).
- Code quality: ESLint + Prettier, enforced via pre-commit hooks and CI.
- API documentation: Swagger/OpenAPI, auto-generated and published per environment.
- Observability: structured JSON logging, centralized log aggregation, application performance monitoring (APM), and uptime alerting.

10. Security Specifications

- Encryption in transit: TLS 1.2+ enforced on all client-to-gateway and service-to-service traffic.
- Encryption at rest: database-level encryption for the storage volume, plus field-level encryption (pgcrypto) for document numbers and other identifying data.
- Attachment access: object-store files are never served directly; access is via short-lived (default 5-minute) signed URLs issued per request.
- RBAC enforcement: every API endpoint checks role and tenant scope at the service layer, not only in the UI; denied requests return 403 with no data leakage.
- Audit logging: all sensitive actions (viewing a full document number, user suspension, configuration changes, escalation dismissal) are recorded in audit_logs with the acting user and timestamp.
- Password & session security: bcrypt-hashed passwords, short-lived access tokens, revocable refresh tokens, and forced logout on password change.
- Data retention: deactivated employee and document records are retained (not hard-deleted) for audit purposes, per the tenant's configured retention policy.
- Vulnerability management: dependency scanning in CI, and periodic third-party penetration testing ahead of major releases (see Section 12).

11. Non-Functional Requirements

Category	Requirement
Availability	Cloud-hosted SaaS supporting 24/7 automated monitoring; target 99.5% monthly uptime for Phase 1.
Performance	Dashboard summary and document list queries return in under 2 seconds for tenants with up to 10,000 documents.
Scalability	Stateless application services scale horizontally; the notification scan job is designed to process at least 100,000 documents per scan cycle without exceeding the cycle interval.
Security	See Section 10 in full.
Usability	Web and mobile interfaces usable by HR and employees with no formal training required.
Compliance	Audit-ready reports generated on demand; all sensitive actions logged.
Configurability	Reminder schedules, templates, document types, and roles configurable per tenant without code changes.
Reliability	Notifications fire accurately and on schedule, with automatic retry on transient delivery failure.

12. Testing Strategy

Test type	Scope
Unit testing	Service-level business logic: status computation, reminder-stage matching, permission checks.
Integration testing	API endpoints against a real database, including multi-tenant isolation checks.
Notification delivery testing	Each channel (email, SMS, push) against provider sandboxes; retry/backoff behavior; duplicate-send prevention.
User acceptance testing (UAT)	Pilot organization exercises real workflows across all four roles against the BRD's success criteria.
Security & penetration testing	Third-party assessment of authentication, RBAC boundaries, attachment access, and tenant isolation.
Load & performance testing	Scan-job throughput and dashboard query latency at representative data volumes.

13. Deployment & Operations

- Environments: isolated dev, staging, and production, each with its own database and configuration.
- CI/CD: GitHub Actions pipeline runs lint, unit, and integration tests on every pull request; staging deploys on merge to main; production deploys via a manual, approved release step.
- Monitoring & alerting: APM for service latency/error rates, uptime checks on the API gateway, and alerting on notification scan-job failures or delivery-provider outages.
- Backup & disaster recovery: automated daily database backups with point-in-time recovery; documented recovery-time and recovery-point objectives reviewed before go-live.
- Rollback: each production deploy is tied to a tagged release, allowing a fast rollback to the previous tag if a regression is detected.

14. Out of Scope (Phase 1)

Restated from Section 1.4 for reference:

- Automated verification or renewal submission with third-party government/regulatory portals.
- Payroll, benefits, or full HRIS functionality beyond document tracking.
- Native offline mode for mobile applications.
- Integration with specific third-party HRIS/ERP systems.
- AI/ML-based document expiry prediction or OCR/auto-extraction.

15. Assumptions & Dependencies

Assumptions

- Organizations onboarding to DocRenew will provide accurate document data and expiry dates for their employees.
- Employees and managers have access to email and/or a mobile device capable of receiving SMS and push notifications.
- The platform is delivered as a cloud-hosted, multi-tenant SaaS product.

Dependencies

- Third-party transactional email provider (e.g. SendGrid or AWS SES).
- Third-party SMS gateway (e.g. Twilio).
- Push notification infrastructure (FCM for Android, APNS for iOS).
- Managed PostgreSQL and Redis hosting.

16. Revision History

Version	Date	Author	Description
0.1	29 Sep 2026	Anil B	Initial draft: Introduction and System Architecture sections.
1.0	13 Oct 2026	Anil B	Completed all sections (Roles, Functional Specs, Notification Design, Data Model, API, UI/UX, Security, NFRs, Testing, Deployment, Scope, Assumptions) for stakeholder review.